

Java

- objektorientierte Sprache
- Syntax ähnlich zu C, C++
- Sammlung standardisierter Bibliotheken für häufige Programmieraufgaben

Syntax von Java:

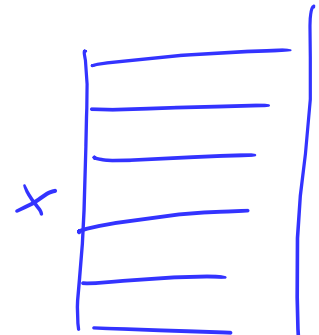
Java Language Specification (Oracle Webseite)

Variablen deklaration

`int x;`

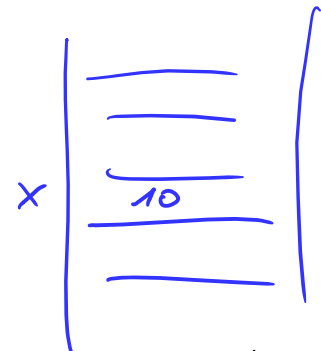
legt einen Speicherbereich für eine int-Zahl an

x ist der symbolische Name der Speicheradresse.



`x = 10;`

Jetzt wird die Zahl 10 an die Speicherzelle x geschrieben.



Variablen deklaration und die Initialisierung können kombiniert werden:

`int x = 10;`

```
int x = 10;
```

Seit Java 10 kann Java den Typ mancher Variablen selbst inferieren:

```
var x = 10;
```

Wir empfehlen die Benutzung von "var" für Prog-Anfänger nicht. Es hilft, wenn man sich über den Typ von Variablen und Ausdrücken im Klaren ist.

Jede Var. muss vor ihrer Benutzung deklariert werden.

Ausführung von Java-Programmen

im JDK (Java Development Kit)

- Java Compiler übersetzt in Java Bytecode.
Dies ist die Maschinsprache der Java Virtual Machine (JVM).
- "java Datei" führt die main-Methode der Datei aus.

Konstanten

"final" bedeutet, dass die Werte der Variablen nur einmal gesetzt und nicht mehr verändert werden

einmal gesetzt und nicht mehr verändert werden
Es ist auch möglich:

```
final int x;
```

```
    ⋮  
    x = 10;
```

```
    ⋮  
x = 11; nicht erlaubt
```

Eingabe und Methodenaufrufe

- Vordef. Klasse IO

- IO.print(wert) : Gibt wert aus, Rückgabe: void
- IO.println(wert) : wie print, aber mit Zeilenumbruch
- IO.println() : nur Zeilenumbruch
- IO.readln(text) : Gibt text aus, liest String ein
- IO.readln() : liest String ein

- KOMMENTARE

```
/*  
    ⋮  
*/
```

lange Kommentare

oder

```
// ... Zeilenende
```

- Bisher: Methodenaufrufe als Anweisung (print, println)
Jetzt auch: Methodenaufrufe als Ausdrücke

Jetzt auch: Methodenaufrufe als Ausdrücke
(`IO.readln`, `Integer.parseInt`, `Math.max`, ...)

- Vordef. Methode `Integer.parseInt` (String) wandelt den String in eine int-Zahl.

z.B. `Integer.parseInt("123")`

liefert die int-Zahl 123

- Übersicht über vordef. Methoden in der Java API (Application Programmer Interface)

Klassen sind organisiert in Paketen (packages)

Pakete — " — Module

Standard-Paket `java.lang`
im Standard-Modul `java.base`

enthält Klassen wie `IO`, `Integer`, `Math`.

Diese kann man direkt verwenden (ohne "import").

Polyorphismus

(Vielgestaltigkeit)

in der Informatik: Eine Operation kann auf Argumente verschiedener Typen angewendet.

Bsp: `IO.print`

- wandelt ihr Argument in String um
mit einer Methode `toString`
- gibt ihn dann aus

Bsp: +

`2 + 3` ist `5`

`"hal" + "lo"` ist `"hallo"`

`"hal" + 2` ist `"hal2"`

`"hal" + 2 + 3` ist `"hal23"`

`"hal" + (2 + 3)` ist `"hal5"`

`2 + 3 + "hal"` ist `"5hal"`

jshell: Read-Evaluate-Print Loop

Tool zur Auswertung von Java-Ausdrücken

Bedingter Ausdruck

$$\begin{array}{ccccc} \text{Aus}_1 & ? & \text{Aus}_2 & : & \text{Aus}_3 \\ \uparrow & & \nwarrow & \nearrow & \\ \text{Typ boolean} & & \text{gleicher Typ} & & \end{array}$$

Erst wird `Aus1` ausgewertet.

Falls sich `true` ergibt, erhält man `Aus2`.

`11. 0 1. - 1. 14 1.`

falls sich true ergibt, erhält man Aus_2 .

Bei false erhält man Aus_3 .

Ausdrücke

- Grundwert (1, 2, true, String (Zeichenkette), ...)
- Name einer Variable (x, y, betrag, ...)

Guter Stil:

Namen von Variablen und Methoden sollten mit Kleinbuchstaben beginnen.

Namen von Dateien (und später Klassen) sollten mit Großbuchstaben beginnen.

- Methodenaufrufe ($Math.max(x, y), \dots$)
- Infix-Operatoren: $+, -, \cdot, \dots$
- Präfix-Operatoren: $+, -, \dots$